

1 Movie Recommendation System with Collaborative Filtering

1.1 Project Objective

In this project, we will build a collaborative filtering recommender system using surprise library. Some of the key highlights of this project are:

1. Use surprise's built-in reader class to process data to work with recommender algorithms
2. Obtain a prediction for a specific user for a particular item
3. Introduce a new user with rating to a rating matrix and make recommendations for them
4. Create a function that will return the top 5 movie recommendations for a user, based on their ratings of other movies

We will be making a movie recommendations based on the MovieLens

<https://grouplens.org/datasets/movielens/latest/>

(<https://grouplens.org/datasets/movielens/latest/>) dataset from the GroupLens research lab at the University of Minnesota.

The MovieLens dataset contains the following files links.csv, movies.csv, ratings.csv and tags.csv. For this project, we will only focus on the movies.csv and ratings.csv to make our recommendations.

1.2 Loading and Preparing the Datasets

```
In [21]: # importing relevant libraries
from surprise.prediction_algorithms import knns
from surprise.similarities import cosine, msd, pearson
from surprise.model_selection import cross_validate
from surprise.prediction_algorithms import SVD
from surprise.prediction_algorithms import KNNWithMeans, KNNBasic, KNNBaseline
from surprise.model_selection import GridSearchCV
import numpy as np
from surprise import accuracy
```

```
In [1]: import pandas as pd

#Load the ratings data

ratings_df = pd.read_csv('ratings.csv')
ratings_df.head()
```

Out[1]:

	userId	movieId	rating	timestamp
0	1	1	4.0	964982703
1	1	3	4.0	964981247
2	1	6	4.0	964982224
3	1	47	5.0	964983815
4	1	50	5.0	964982931

```
In [2]: ratings_df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 100836 entries, 0 to 100835
Data columns (total 4 columns):
#   Column      Non-Null Count  Dtype
---  ---
0   userId      100836 non-null  int64
1   movieId     100836 non-null  int64
2   rating      100836 non-null  float64
3   timestamp   100836 non-null  int64
dtypes: float64(1), int64(3)
memory usage: 3.1 MB
```

1.2.1 Observation 1

The ratings dataset has 100836 rows and 4 columns. Another observation is that there are no null/missing therefore we will not be cleaning the dataset.

We are only interested in the userID, movieId and rating therefore we drop the timestamp column.

```
In [3]: ratings_df = ratings_df.drop(columns=['timestamp'], axis=1)
ratings_df.head()
```

Out[3]:

	userId	movieId	rating
0	1	1	4.0
1	1	3	4.0
2	1	6	4.0
3	1	47	5.0
4	1	50	5.0

```
In [4]: #Load the movies data
movies_df = pd.read_csv('movies.csv')
movies_df.head()
```

Out[4]:

	movieId	title	genres
0	1	Toy Story (1995)	Adventure Animation Children Comedy Fantasy
1	2	Jumanji (1995)	Adventure Children Fantasy
2	3	Grumpier Old Men (1995)	Comedy Romance
3	4	Waiting to Exhale (1995)	Comedy Drama Romance
4	5	Father of the Bride Part II (1995)	Comedy

```
In [5]: movies_df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 9742 entries, 0 to 9741
Data columns (total 3 columns):
#   Column   Non-Null Count  Dtype
---  -
0   movieId  9742 non-null   int64
1   title    9742 non-null   object
2   genres   9742 non-null   object
dtypes: int64(1), object(2)
memory usage: 228.5+ KB
```

1.2.2 Observation 2

The movies dataset has 9742 rows and 3 columns. Another observation is that there are no null/missing therefore we will not be cleaning the dataset.

1.2.3 Merge the ratings and Movies Datasets

1. movieId is common in both the ratings and movie datasets

2. merge the ratings and movies datasets to add the movie title and genres to the ratings dataset

In [8]:

```
new_df = pd.merge(ratings_df, movies_df, on='movieId')
new_df.head()
```

Out[8]:

	userId	movieId	rating	title	genres
0	1	1	4.0	Toy Story (1995)	Adventure Animation Children Comedy Fantasy
1	5	1	4.0	Toy Story (1995)	Adventure Animation Children Comedy Fantasy
2	7	1	4.5	Toy Story (1995)	Adventure Animation Children Comedy Fantasy
3	15	1	2.5	Toy Story (1995)	Adventure Animation Children Comedy Fantasy
4	17	1	4.5	Toy Story (1995)	Adventure Animation Children Comedy Fantasy

1.2.4 Preparing data for surprise

Here we transform the dataset into something compatible with surprise. In order to do this, you're going to need Reader and Dataset classes

In [9]:

```
from surprise import Dataset, Reader

# Define the rating scale that is from 0.5 to 5.0
reader = Reader(rating_scale=(0.5, 5.0))

#Convert the Dataframe to a Surprise dataset
data = Dataset.load_from_df(new_df[['userId', 'movieId', 'rating']], reader)
```

1.2.5 Split the Data

Split the data into training set(80% to be used to train the recommendation model) and testset(20% to be used to evaluate the model's performance)

In [10]:

```
from surprise.model_selection import train_test_split

trainset, testset = train_test_split(data, test_size=0.2, random_state=42)
```

1.3 Determine the Best Model

1.3.1 Check item-item similarity versus user-user similarity.

For the sake of computation time, it's best to calculate the similarity between whichever number is fewer, users or items

In [11]:

```
print("Number of users: ", trainset.n_users, "\n")
print("Number of items: ", trainset.n_items, "\n")
```

Number of users: 610

Number of items: 8982

1.3.2 Observation

There are more items than users therefore it will be more efficient to calculate user-user similarity as opposed to item to item similarity

1.3.3 Memory Based/Neighbourhood Models

We start with the memory based/neighbourhood models and use RMSE to test the model predictions. The lower the values of RMSE the better the model

1. Approach 1: Check which is the better similarity matrix i.e cosine or pearson
2. Approach 2: Apply the better performing similarity matrix to KNNBasic, KNNBaseline, KNNWithMeans

The one with lowest RMSE will be the best performing model under memory based/neighbourhood models

In [22]:

```
#KNNBasic with cosine
sim_cos = {"name": "cosine", "user_based": False}
basic = knns.KNNBasic(sim_options=sim_cos)
basic.fit(trainset)
predictions = basic.test(testset)
print(accuracy.rmse(predictions))
```

Computing the cosine similarity matrix...

Done computing similarity matrix.

RMSE: 0.9813

0.9813236683254013

```
In [24]: #KNNBasic with pearson
sim_pearson = {"name": "pearson", "user_based": False}
basic_pearson = knns.KNNBasic(sim_options=sim_pearson)
basic_pearson.fit(trainset)
predictions = basic_pearson.test(testset)
print(accuracy.rmse(predictions))
```

```
Computing the pearson similarity matrix...
Done computing similarity matrix.
RMSE: 0.9731
0.9731262979520426
```

1.3.4 Observation

The pearson similarity matrix is performing better than the cosine one so we apply pearson to KNNBaseline and KNNWithMeans

```
In [26]: #KNNWithMeans with pearson
sim_pearson = {"name": "pearson", "user_based": False}
knn_means = knns.KNNWithMeans(sim_options=sim_pearson)
knn_means.fit(trainset)
predictions = knn_means.test(testset)
print(accuracy.rmse(predictions))
```

```
Computing the pearson similarity matrix...
Done computing similarity matrix.
RMSE: 0.9085
0.9084653777602131
```

```
In [27]: #KNNBaseline with pearson
sim_pearson = {"name": "pearson", "user_based": False}
knn_baseline = knns.KNNBaseline(sim_options=sim_pearson)
knn_baseline.fit(trainset)
predictions = knn_baseline.test(testset)
print(accuracy.rmse(predictions))
```

```
Estimating biases using als...
Computing the pearson similarity matrix...
Done computing similarity matrix.
RMSE: 0.8819
0.8818517179287566
```

1.3.5 Observation

From RMSE values from KNNBasic, KNNWithMeans and KNNBaseline the best model is KNNBaseline

1.3.6 Model Based

The next approach to determine the best model is to try out the Model based method with SVD(Singular Value Decomposition)

```
In [13]: #perform a gridsearch with SVD

params = {'n_factors': [20, 50, 100],
          'reg_all': [0.02, 0.05, 0.1]}

grid_svd = GridSearchCV(SVD, param_grid=params, n_jobs=-1)
grid_svd.fit(data)
```

```
In [14]: best_params = grid_svd.best_params['rmse']
best_params
```

```
Out[14]: {'n_factors': 100, 'reg_all': 0.05}
```

```
In [17]: svd = SVD(n_factors=100, reg_all=0.05)
svd.fit(trainset)
predictions = svd.test(testset)
print(accuracy.rmse(predictions))
```

```
RMSE: 0.8688
0.8688082598324457
```

1.3.7 Observation

TeE SVD has an even lower RMSE value than KNNBaseline therefore we will proceed to use SVD to make recommendations

1.4 Making Recommendations

The process of making the recommendations is as follows:

1. Create a list of user rating for a new user
2. Make predictions with new ratings
3. Order the predictions from highest to lowest rated
4. return the top 5 recommendations

```
In [31]: #Step1 Create a list of user rating for a new user

user_rating = [
    {'userId': 2999, 'movieId': 356, 'rating': '4'},
    {'userId': 2999, 'movieId': 318, 'rating': '3'},
    {'userId': 2999, 'movieId': 593, 'rating': '2'},
    {'userId': 2999, 'movieId': 2571, 'rating': '5'},
]
```

In [32]: *#Step2 make predictions with new ratings*

```
user_ratings = pd.DataFrame(user_rating)
new_ratings_df = pd.concat([ratings_df, user_ratings], axis=0)
brand_new_df = pd.merge(new_ratings_df, movies_df, on='movieId')
brand_new_df.head()
```

Out[32]:

	userId	movieId	rating	title	genres
0	1	1	4.0	Toy Story (1995)	Adventure Animation Children Comedy Fantasy
1	5	1	4.0	Toy Story (1995)	Adventure Animation Children Comedy Fantasy
2	7	1	4.5	Toy Story (1995)	Adventure Animation Children Comedy Fantasy
3	15	1	2.5	Toy Story (1995)	Adventure Animation Children Comedy Fantasy
4	17	1	4.5	Toy Story (1995)	Adventure Animation Children Comedy Fantasy

In [33]: *#repeat the steps above for preparing data for surprise and spliting the data*
#Define the rating scale that is from 0.5 to 5.0

```
reader = Reader(rating_scale=(0.5, 5.0))
```

#Convert the Dataframe to a Surprise dataset

```
new_data = Dataset.load_from_df(brand_new_df[['userId', 'movieId', 'rating']],
```

#Split the data

```
trainset, testset = train_test_split(new_data, test_size=0.2, random_state=42)
```

In [34]: *#train the model using SVD*

```
svd2 = SVD(n_factors=100, reg_all=0.05)
svd2.fit(trainset)
```

Out[34]: <surprise.prediction_algorithms.matrix_factorization.SVD at 0x2008930c350>

In [35]: *#make predictions for the user*

```
list_of_movies = []
for m_id in brand_new_df['movieId'].unique():
    list_of_movies.append( (m_id, svd2.predict(2999, m_id)[3]))
```

In [36]: *#Step3 Order the predictions from highest to lowest rated*

```
ranked_movies = sorted(list_of_movies, key=lambda x:x[1], reverse=True)
```


In [37]: *#Step4 return the top n recommendations*

```
def recommended_movies(user_ratings, movie_title_df, n):
    for idx, rec in enumerate(user_ratings):
        title = movie_title_df.loc[movie_title_df['movieId'] == int(rec[0])]['title'].values[0]
        print('Recommendation # ', idx+1, ': ', title, '\n')
        n -= 1
        if n == 0:
            break

recommended_movies(ranked_movies, movies_df, 5)
```

Recommendation # 1 : 916 Army of Darkness (1993)

Name: title, dtype: object

Recommendation # 2 : 933 Boot, Das (Boat, The) (1981)

Name: title, dtype: object

Recommendation # 3 : 602 Dr. Strangelove or: How I Learned to Stop Wor
r...

Name: title, dtype: object

Recommendation # 4 : 1939 Matrix, The (1999)

Name: title, dtype: object

Recommendation # 5 : 7355 Toy Story 3 (2010)

Name: title, dtype: object

In []: